

501: Linux Operating System [2511000905044001]

Unit 3 : Shell Scripting in Linux

- 3.1 Creating and Executing Shell Scripts (nano, vi, ./script.sh)
- 3.2 Shell Metacharacters and Operators
 - 3.2.1 Filename Expansion (wildcards: *, ?, [])
 - 3.2.2 Input/Output Redirection (>, >>, <)
 - 3.2.3 Pipes (|)
 - 3.2.4 Command Substitution (\${...}, ...)
- 3.3 Control Flow Structures (if-else, case, for, while, until)
- 3.4 Logical Operators (&&, ||, !)
- 3.5 test and [] command for Condition Testing (file, numeric, string)
- 3.6 Arithmetic Operations (expr, \$(()))

Creating and Executing Shell Scripts:

What is a Shell Script?

- A shell script is a **text file** containing a sequence of **commands** for the shell to execute.
- It automates repetitive tasks like backups, software installation, etc.
- Written in **shell languages** like Bash, sh, etc.

Creating a Shell Script File

1. Create a shell script file:

```
vi myscript.sh
```

2. Press i to enter insert mode.

3. Type the script:

```
echo "Hello, welcome to shell scripting!"
```

4. Save and exit:

- Press Esc key
- Then type :wq and press Enter

5. Make script executable:

```
chmod +x myscript.sh
```

6. Run the script:

```
./myscript.sh
```

What Are Shell Meta characters?

Metacharacters are **special symbols** in the shell that have **specific meanings**. They help in tasks like redirection, piping, wildcards, background processes, etc.

1. Common Shell Metacharacters :

Metacharacter	Meaning / Use	Example
*	Matches zero or more characters	ls *.txt (lists all .txt)
?	Matches exactly one character	ls file?.txt
[]	Matches a range or set of characters	ls file[1-3].txt
\	Escape a metacharacter	echo \$HOME
;	Separates multiple commands	date ; whoami
&&	Runs second command only if first succeeds	mkdir test && cd test
`		`
&	Run process in background	sleep 10 &
	` (pipe)	Send output of one command to another command
>	Redirect output to a file (overwrite)	echo "Hi" > file.txt
>>	Append output to file	echo "Hello" >> file.txt
<	Take input from file	wc -l < file.txt

2. Shell Operators :

Shell operators are used in scripting for **comparison, assignment, logical and arithmetic operations.**

a) Arithmetic Operators

Operator	Description	Example
+	Addition	echo \$((3 + 2))
-	Subtraction	echo \$((5 - 1))
*	Multiplication	echo \$((2 * 3))
/	Division	echo \$((10 / 2))
%	Modulus (remainder)	echo \$((7 % 3))

b) Comparison Operators (for numbers)

Used with [] or test command.

Operator	Meaning	Example
-eq	Equal to	[5 -eq 5]
-ne	Not equal to	[4 -ne 2]
-gt	Greater than	[6 -gt 2]
-lt	Less than	[1 -lt 9]
-ge	Greater or equal	[7 -ge 7]
-le	Less or equal	[2 -le 4]

c) String Operators

Operator	Description	Example
=	Strings are equal	["\$a" = "\$b"]
!=	Strings are not equal	["\$a" != "\$b"]
-z	String is empty	[-z "\$name"]
-n	String is not empty	[-n "\$name"]

d) Logical Operators

Operator	Meaning	Example
!	NOT	if [! -f file.txt]
-a	AND	[-f file -a -r file]
-o	OR	[-f file -o -r file]

Example Script Using Metacharacters and Operators :

```
echo "Enter a number:"  
read num
```

```
if [ $num -gt 10 ]; then  
    echo "Greater than 10"  
else  
    echo "10 or less"  
fi
```

Filename Expansion in Shell (Wildcards: *, ?, []):

What is Filename Expansion?

In the shell (like bash), **filename expansion** means using **wildcards (also called globbing patterns)** to match multiple files or directories without typing their full names.

Wildcards are **special characters** used to search or specify patterns in filenames.

1. * (Asterisk – Match All)

Matches: zero or more characters

Examples:

Command	Meaning/Matches
ls *.txt	All files ending with .txt
ls a*	All files starting with a
ls *	Lists all files and folders in the directory

2. ? (Question Mark – Match Single Character)

Matches: exactly one character

Examples:

Command	Meaning/Matches
ls file?.txt	Matches file1.txt, file2.txt, etc.
ls a?b.txt	Matches a1b.txt, a_b.txt, acb.txt

3. [] (Square Brackets – Match Range/Set)

Matches: any one character from the set/range inside

Examples:

Command	Meaning/Matches
ls file[1-3].txt	Matches file1.txt, file2.txt, file3.txt
ls file[a-c].txt	Matches filea.txt, fileb.txt, filec.txt
ls file[135].txt	Matches file1.txt, file3.txt, file5.txt

Combine Wildcards

You can use multiple wildcards together.

Examples:

```
ls a*.sh    # All .sh files starting with a
ls test??.c # Files like test01.c, testab.c (2 characters after 'test')
ls file[1-5]* # Files starting with file1 to file5
```

Notes :

- Wildcards work with **file and directory names only**, not contents.
- Shell expands the pattern **before** running the command.
- You can use echo to test wildcards: echo *.txt
- Use quotes to prevent wildcard expansion: echo "*.txt" prints *.txt

Example:

Assume your directory has these files:

report1.txt, report2.txt, report3.csv, result1.doc, result2.doc

- ls *.txt
- ls report?.txt
- ls result[12].doc

Input/Output Redirection in Shell :

What is Redirection?

- In shell, **redirection** means **sending output to a file** or **reading input from a file**.

- You use special symbols like `>`, `>>`, `<` to redirect input/output streams.

1. Output Redirection: `>`

- Used to **redirect standard output (stdout)** to a file.
- If the file **already exists**, it will be **overwritten**.

Syntax:

`command > filename`

Example:

```
echo "Hello" > output.txt
```

➡Creates a file `output.txt` and writes `Hello` into it (deletes previous content if file exists).

2. Append Output: `>>`

- Used to **append** output to the end of the file.
- Keeps existing content and **adds new content at the bottom**.

Syntax:

`command >> filename`

Example:

```
echo "New line" >> output.txt
```

➡Adds New line at the end of `output.txt`.

3. Input Redirection: `<`

- Used to **take input from a file** instead of keyboard.
- Passes file content as input to a command.

Syntax:

`command < filename`

Example:

wc -l < input.txt

➡ Counts number of lines in input.txt.

Additional Examples

Command	Description
cat > file.txt	Creates or overwrites file.txt with typed input
cat >> file.txt	Appends new text to file.txt
sort < names.txt	Sorts the contents of names.txt
grep "test" < file.txt	Searches for "test" in file.txt

Important Notes

- Use > carefully — it will **erase** existing file content.
- Use >> to **safely append** without removing old data.
- < is helpful when a program expects input from a user or file.
- Combine with pipes (|) for more advanced redirection.

Table:

Symbol	Use	Action
>	Output Redirection	Overwrites file
>>	Append Output	Adds to the end of file
<	Input Redirection	Reads input from file

Pipes (|) in Shell:

What is a Pipe?

- A **pipe (|)** is used to **pass the output of one command as input to another**.
- It connects **two or more commands** together.
- Pipes help to **chain commands** for efficient data processing.

Syntax

command1 | command2

➡ The **output** of command1 becomes the **input** of command2.

Key Points

- Used to **combine commands**.
- No temporary file is created.
- It works with **standard output** and **standard input** (stdout → stdin).
- Multiple pipes can be used in one line.

Key Points:

1. **Connects Commands:**
 - Pipes allow **multiple commands** to be combined into a **single line**.
2. **Used in Shell Scripting and Terminal:**
 - Frequently used in **bash**, **sh**, and **other shells** for data processing.
3. **Standard Output and Input:**
 - Pipe works by linking the **stdout (standard output)** of one command to the **stdin (standard input)** of another.
4. **Does Not Use Intermediate File:**
 - The transfer of data happens **directly in memory**, saving time and space.
5. **Processes Streamed Data:**
 - Useful in working with **large outputs** or continuous streams (like logs).

Examples:

1. **List Files and Count:**

```
ls | wc -l
```

Lists files and counts how many are there.

2. **Search in Files:**

```
cat file.txt | grep "keyword"
```

Searches for “keyword” in file.txt.

3. **Sort and Remove Duplicates:**

```
cat names.txt | sort | uniq
```

Sorts the lines and removes duplicates.

4. See Running Processes:

```
ps aux | grep chrome
```

Filters chrome related processes.

5. Long Listing Sorted by Size:

```
ls -l | sort -k5 -n
```

Lists files and sorts by file size.

Command	Purpose
grep	Filter text
wc	Count lines, words, characters
sort	Sort text
uniq	Remove duplicates
cut	Cut parts of text
tee	Save and display output

Command Substitution (\$(...), ...) :

- **Command substitution** allows the **output of a command** to replace the **command itself** in a shell script or command line.
- It's used when you want to **use the result of one command inside another**.

Write a shell script to draw following pattern.

```
*  
* *  
* * *  
* * * *  
* * * * *
```

Step :1	vi star_pattern.sh
Step :2	Press i to enter INSERT mode
Step :3	<pre>rows=5 for ((i=1; i<=rows; i++)) do for ((j=1; j<=i; j++)) do echo -n "*" done echo done</pre>
Step :4	Save and exit: Press Esc key (to exit insert mode) Type : wq and press Enter
Step :5	chmod +x star_pattern.sh
Step :6	./star_pattern.sh

Write a shell script to find all prime numbers in given range.

Step :1	vi prime_range.sh
Step :2	Press i to enter INSERT mode
Step :3	<pre>read -p "Enter starting number: " start read -p "Enter ending number: " end echo "Prime numbers between \$start and \$end are:" for ((num=start; num<=end; num++)) do if [\$num -le 1]; then continue fi is_prime=1 for ((i=2; i*i<=num; i++))</pre>

	<pre> do if ((num % i == 0)); then is_prime=0 break fi done if [\$is_prime -eq 1]; then echo -n "\$num " fi done echo </pre>
Step :4	Save and exit: Press Esc key (to exit insert mode) Type : wq and press Enter
Step :5	chmod +x prime_range.sh
Step :6	./ prime_range.sh

Write a shell script to find sum of digits of a number

Step :1	vi sumdigit.sh
Step :2	Press i to enter INSERT mode
Step :3	<pre> read -p "Enter a number: " num sum=0 n=\$num # Loop to extract and add digits while [\$n -gt 0] do digit=\$((n % 10)) # Get the last digit sum=\$((sum + digit)) # Add it to sum n=\$((n / 10)) # Remove the last digit done echo "Sum of digits of \$num is: \$sum" </pre>
Step :4	Save and exit:

	Press Esc key (to exit insert mode) Type : wq and press Enter
Step :5	chmod +x sumdigit.sh
Step :6	./ sumdigit.sh

Write a shell script to print fibonacci series upto entered value N.

Step :1	vi fibonacci.sh
Step :2	Press i to enter INSERT mode
Step :3	<pre># Read the limit N from the user read -p "Enter the maximum value N: " N # Initialize first two terms a=0 b=1 echo "Fibonacci series up to \$N:" echo -n "\$a \$b " # Generate next terms until the value exceeds N while true do fib=\$((a + b)) if [\$fib -gt \$N]; then break fi echo -n "\$fib " # Shift values a=\$b b=\$fib done echo</pre>
Step :4	Save and exit: Press Esc key (to exit insert mode) Type : wq and press Enter

Step :5	chmod +x fibonacci.sh
Step :6	./ fibonacci.sh

Command Substitution (\$(...), ...) :

What is Command Substitution?

Command substitution lets you **use the output of a command as input** to another command.

Examples:

1. Display current date:

```
echo "Today is: $(date)"
```

2. Save command output in a variable:

```
mydir=$(pwd)
echo "Current directory is $mydir"
```

3. Count files in a folder:

```
echo "Files: $(ls | wc -l)"
```

4. Nesting (only easy with \$(...)):

```
echo "Time: $(echo $(date))"
```

Control Flow Structures (if-else, case, for, while, until) :

Control flow allows scripts to **make decisions** and **repeat actions** — essential for automation.

1. if, else, elif — Conditional branching

Syntax:

```
if [ condition ]; then
    # commands if true
elif [ other_condition ]; then
    # commands if elif is true
else
    # commands if all false
fi
```

Example:

```
num=10
if [ $num -gt 0 ]; then
    echo "Positive"
else
    echo "Zero or Negative"
fi
```

2. case — Multi-way branching (switch)**Syntax:**

```
case $variable in
    pattern1)
        # commands
        ;;
    pattern2)
        # commands
        ;;
    *)
        # default
        ;;
esac
```

Example:

```
fruit="apple"
case $fruit in
    apple) echo "It's an apple";;
    banana) echo "It's a banana";;
    *) echo "Unknown fruit";;
esac
```

3. for loop — Loop over a list**Syntax:**

```
for item in list; do
    # commands
done
```

Example:

```
for name in Alice Bob Charlie; do
    echo "Hello $name"
```

done

4. while loop — Run while condition is true

Syntax:

```
while [ condition ]; do
    # commands
done
```

Example:

```
count=1
while [ $count -le 5 ]; do
    echo "Count: $count"
    count=$((count + 1))
done
```

5. until loop — Run until condition becomes true

Syntax:

```
until [ condition ]; do
    # commands
done
```

Example:

```
count=1
until [ $count -gt 5 ]; do
    echo "Count: $count"
    count=$((count + 1))
done
```

Table :

Structure	Purpose	Ends With
if-else	Conditional branching	fi
case	Multiple condition check	esac
for	Loop over values	done

Structure	Purpose	Ends With
while	Loop while condition is true	done
until	Loop until condition is true	done

Logical Operators (&&, ||, !):

Logical operators are used in shell scripting to **combine multiple conditions**. They return **true (0)** or **false (non-zero)** and help control the flow of execution (like in if, while, etc.).

1. AND Operator &&

- **Syntax:**

command1 && command2

- **Explanation:** If command1 is successful (exit status 0), then command2 runs.
- **Example:**

mkdir test_dir && cd test_dir

- cd test_dir runs **only if** mkdir test_dir succeeds.

2. OR Operator ||

- **Syntax:**

command1 || command2

- **Explanation:** If command1 **fails**, then command2 runs.
- **Example:**

cd myfolder || echo "Folder not found"

- If cd fails (folder doesn't exist), it prints the error message.

3. NOT Operator !

- **Syntax:**

```
if ! command; then
    # do something
fi
```

- **Explanation:** Runs the block if the command **fails**.

- **Example:**

```
if ! grep "hello" file.txt; then
    echo "Text not found"
fi
```

- echo runs if grep does **not** find the text.

test and [] command for Condition Testing (file, numeric, string)

test and [] are used to **evaluate conditional expressions**. Both are almost the same:

```
test expression    # Old syntax
[ expression ]     # Preferred and more common
```

1. File Condition Testing

Expression	Meaning
-e file	File exists
-f file	File exists and is a regular file
-d file	File exists and is a directory
-r file	File is readable
-w file	File is writable
-x file	File is executable
-s file	File exists and is not empty

Example:

```
if [ -f "data.txt" ]; then
    echo "File exists"
fi
```

2. Numeric Condition Testing

Expression	Meaning
n1 -eq n2	Equal to
n1 -ne n2	Not equal to
n1 -gt n2	Greater than
n1 -lt n2	Less than
n1 -ge n2	Greater than or equal to
n1 -le n2	Less than or equal to

Example:

```
a=10
b=20
if [ $a -lt $b ]; then
    echo "a is less than b"
fi
```

3. String Condition Testing

Expression	Meaning
str1 = str2	Strings are equal
str1 != str2	Strings are not equal
-z str	String is empty
-n str	String is not empty

Example:

```
name="Jaimini"
if [ "$name" = "Jaimini" ]; then
    echo "Name is Jaimini"
fi
```

Notes: Always quote variables in ["\$var" = "value"] to avoid errors.

Logical Operators with []

You can combine conditions:

```
if [ -f file.txt ] && [ -s file.txt ]; then
    echo "File exists and is not empty"
fi
```

Arithmetic Operations in Shell

you can perform arithmetic operations using:

1. **expr command (older method)**
2. **\$(()) (preferred modern syntax)**

1. Using expr

- expr evaluates an arithmetic expression and returns the result.

Syntax:

expr operand1 operator operand2

Example:

```
a=10
b=5
result=`expr $a + $b`
echo "Sum: $result"
```

Note: Use backticks `` or \$() for command substitution.

Important:

- **Spaces are mandatory** between numbers and operators.
- * must be escaped with * to avoid shell expansion.

```
expr $a \* $b # Multiplication
```

2. Using \$(()) – Recommended

- This is **arithmetic expansion** syntax in Bash.
- Easier, cleaner, and more powerful.

Syntax:

\$((expression))

Example:

```
a=10
```

```
b=5
sum=$((a + b))
echo "Sum: $sum"
```

```
product=$((a * b))
echo "Product: $product"
```

No need for escape characters or spaces.

Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
**	Power (only in some shells like Bash 4+)

Increment & Decrement

```
a=5
((a++)) # Post-increment
((++a)) # Pre-increment
```

```
((a--)) # Post-decrement
((--a)) # Pre-decrement
```

Example Script:

```
a=20
b=4

echo "Addition: $((a + b))"
echo "Subtraction: $((a - b))"
echo "Multiplication: $((a * b))"
echo "Division: $((a / b))"
```

```
echo "Modulus: $((a % b))"
```